

プログラミングI 演習

Computer Programming I: Exercises

第7回 ポインタ

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

本日の講義

本日の目標（今回からポインタを使うので難易度があがります）

- ポインタとその利点について知ろう
- ポインタの使い方を覚えよう
- ポインタを使った簡単なプログラムを作成・実行してみよう

本日の資料

1. 前回のおさらい
2. ポインタ
3. 演習課題

1. 前回のおさらい

数学での関数

$$f(x) = 7.2x + 3.0$$

変数「x」に値を代入することで
ある値 $f(x)$ が決定する

Cでの関数(function)

```
double function(double x)
{
    return 7.2*x+3.0;
}
```

```
int main(void)
{
    double fx = function(10.0);
    return 0;
}
```

ある処理をまとめたもの

プログラム中の特定の処理を関数化し、
プログラムを短く読みやすくなる

実は, printf() も関数の1つ。ただし、
数十行のプログラムからできている。
もし関数がいなければ、これまでの演習のプログラム
の幾つかは数百行になっていた。

3

1. 前回のおさらい

関数の定義方法

戻り値の型 関数名(引数リスト)

```
{
    処理
    return 戻り値;
}
```

```
double function(double x)
```

```
{
    return 7.2*x+3.0;
}
```

引数(ひきすう)

- 関数内の処理に必要な様々なデータを受け取るための変数
- 複数の引数を定義することができる
- 引数に配列を使う方法は第8回

戻り値とreturn文

- 関数での結果を1つだけ返すことができる
- この値を戻り値と呼ぶ
- 戻り値は「return」文によって設定する
- 「return」文の実行直後に関数を抜ける

【その他の注意点】

- 戻り値の型は必ず正しく指定する
- 関数の中で別の関数を定義することはできない。呼び出すことはできる。

4

1. 前回のおさらい

stdio.h, stdlib.h, math.hはヘッダファイルと呼ばれ、そこで多くの関数がプロトタイプ宣言されている

関数の定義位置

関数は使用前に宣言する必要がある
下記のプログラムはエラーとなる
(WARNINGになる場合もある)

```
int main(void)
{
    double fx = function(10.0);;
    return 0;
}

double function(double x)
{
    return 7.2*x+3.0;
}
```

エラー: 未定義な関数です!
implicit declaration of function



プロトタイプ宣言

関数の形式(戻り値と引数)だけを先に
コンパイラに教えておく
関数の定義位置は自由に変えられる

```
double function(double x);

int main(void)
{
    double fx = function(10.0);
    return 0;
}

double function(double x)
{
    return 7.2*x+3.0;
}
```

5

1. 前回のおさらい

変数の局所性

変数「x」という名前が使われている場所

↓main関数とSub関数で同じ名の変数がある

```
double Sub(double x, double y)
{
    x = x - y;      変数「x」を使って計算
    return x;
}

int main(void)
{
    double x=5.0, y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x, y));
    printf("x=%f\n", x);
    return 0;
}
```

関数内でxの値を変更してしまっているが、呼び出し側(main)に影響はないのだろうか?



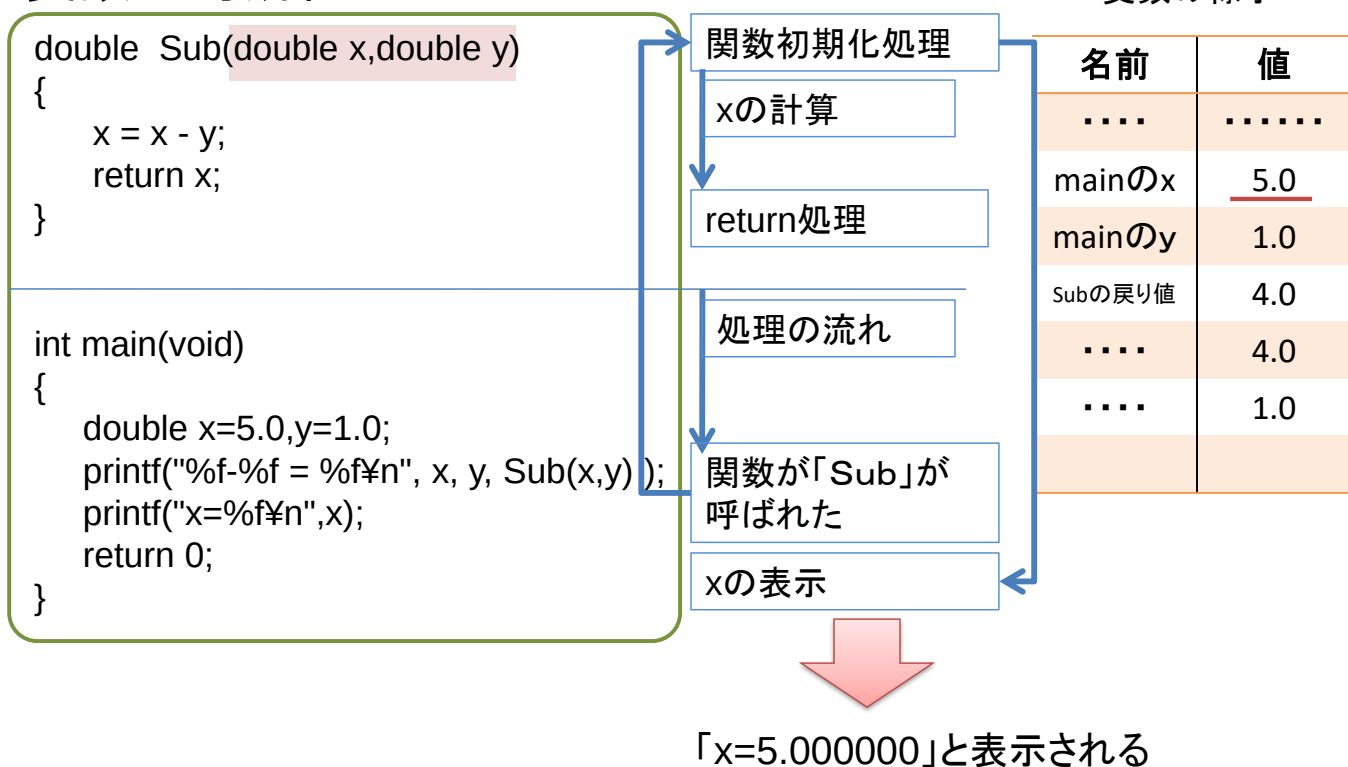
引数や関数内で宣言した変数は関数毎に独立に存在している

「x=5.000000」と表示され、Subにおけるxに関する計算結果は、mainのxには反映されない!

6

1. 前回のおさらい

変数の局所性



7

1. 前回のおさらい

演習6-3 関数の作成練習①

整数a, bを入力し, aのb乗を計算する関数powerを作成し, 実行してみよう.
例: a=3, b=3のとき, 関数powerの引数としてa, bを読み, aのb乗を戻り値にする

5の2i+1乗(マチンの公式): 演習課題4-5

```
for(j=1, n_jou=1; j<=2*i+1; j++) {n_jou = n_jou * 5;}
```

3のn乗(数列): 演習課題5-3[2]

```
n_jou = 1;
for(i=1; i<=n; i++) {n_jou = n_jou * 3;}
```

C言語にはべき乗を計算する関数powが用意されています.
double pow(double x, double y);
math.h をインクルードすると利用できます.
自作の関数powerと, powによる計算結果を比較しましょう.

8

1. 前回のおさらい

演習6-3 解答例

```
#include<stdio.h>
int power(int a, int b);      /* プロトタイプ宣言 */

int main(void)
{
    int a, b, ans;
    printf("aを入力:");      scanf("%d", &a);
    printf("bを入力:");      scanf("%d", &b);
    ans = power(a, b);
    printf("%dの%d乗は%dです\n", a, b, ans);
    return 0;
}

int power(int a, int b){
    int i, ans = 1;
    for(i=1; i<=b; i++){
        ans = ans * a;
    }
    return ans;
}
```

9

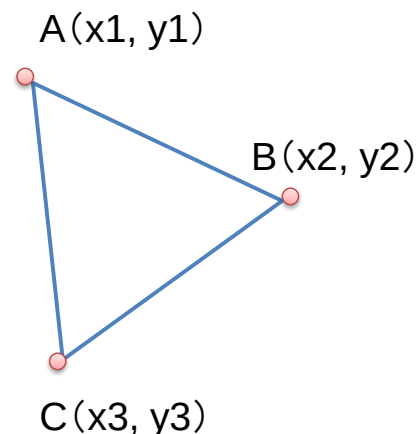
1. 前回のおさらい

発展6-1 関数の作成練習②: 三角形の面積を求める関数

XY平面上の3点A, B, Cについて, 各点のx座標, y座標を入力し, 三角形ABCの面積を求める関数を作成してみよう.

```
double GetArea3(double x1, double y1,
                 double x2, double y2,
                 double x3, double y3)
{
    .....
}
```

$$\text{面積} = \frac{1}{2} |AB| |AC| \sin(\angle CAB)$$



※ヒント: 以下の値が必要

- $\sin \angle CAB$ → 右参照
- 各辺の長さ → 次のスライド参照

余弦定理でcosを求め, そこからsinを計算

$$|BC|^2 = |AB|^2 + |AC|^2 - 2|AB||AC|\cos(\angle CAB)$$

10

1. 前回のおさらい

発展6-1 解答例(1/3: プロトタイプ宣言とmain関数)

```
#include<stdio.h>
#include<math.h>
double GetLength(double x1, double y1, double x2, double y2);
double GetArea3 (double x1, double y1, double x2, double y2,
                  double x3, double y3);

int main(void)
{
    double ax, ay, bx, by, cx, cy;
    printf("点Aのx座標¥n"); scanf("%lf", &ax);
    printf("点Aのy座標¥n"); scanf("%lf", &ay);
    printf("点Bのx座標¥n"); scanf("%lf", &bx);
    printf("点Bのy座標¥n"); scanf("%lf", &by);
    printf("点Cのx座標¥n"); scanf("%lf", &cx);
    printf("点Cのy座標¥n"); scanf("%lf", &cy);
    printf("¥n△ABCの面積は%lf", GetArea3(ax, ay, bx, by, cx, cy));
    return 0;
}
```

※GetLength関数は、プログラム I 講義のテキスト6.7節の演習課題1にあるeuclideanDist関数とほぼ同じ(ただし引数の型が違う)

11

1. 前回のおさらい

発展6-1 解答例(2/3: GetLength関数)

```
double GetLength(double x1, double y1, double x2, double y2){
    /* 長さの公式 */
    return sqrt( (x1 - x2)*(x1 - x2) + (y1 - y2)*(y1 - y2) );
}
```

GetArea3関数の考え方

$$\triangle ABC \text{の面積: } \frac{1}{2} |AB| |AC| \sin(\angle CAB)$$

$$2 \text{点 } (x_1, y_1), (x_2, y_2) \text{間の距離: } \sqrt{|x_1 - x_2|^2 + |y_1 - y_2|^2}$$

$$\text{余弦定理: } |BC|^2 = |AB|^2 + |AC|^2 - 2|AB||AC|\cos(\angle CAB)$$

$$\rightarrow \cos(\angle CAB) = \frac{|AB|^2 + |AC|^2 - |BC|^2}{2|AB||AC|}$$

$$\cos \angle A \text{から} \sin \angle A \text{への変換: } \sin \angle A = \pm \sqrt{1 - \cos^2 A}$$

→ $\angle A$ は三角形の内角で $0^\circ < \angle A < 180^\circ$ なので, $\sin \angle A$ は必ず正になる

12

1. 前回のおさらい

発展6-1 解答例(3/3: GetArea3関数)

```
double GetArea3(double x1, double y1, double x2, double y2,  
                double x3, double y3){  
    double Area, CosCAB, SinCAB, LengthAB, LengthBC, LengthAC;  
  
    LengthAB = GetLength(x1,y1,x2,y2);    /* 辺ABの長さ */  
    LengthBC = GetLength(x2,y2,x3,y3);    /* 辺BCの長さ */  
    LengthAC = GetLength(x1,y1,x3,y3);    /* 辺ACの長さ */  
  
    /* 余弦定理 */  
    CosCAB = (LengthAB*LengthAB + LengthAC*LengthAC -  
              LengthBC*LengthBC) / (2.0*LengthAB*LengthAC);  
  
    /*Sinを計算*/  
    SinCAB = sqrt(1.0 - CosCAB*CosCAB);  
  
    /*面積の式*/  
    Area = LengthAB * LengthAC * SinCAB / 2.0;  
    return Area;  
}
```

13

2. ポインタ

C言語の大事な概念で、身に着けると出来ることが増えます

- **ポインタ/ポインタ変数: 「アドレス」を保存するための変数**
- 関数に配列を引数として渡すとき, 「値渡し」「ポインタ渡し」の違いを知っておく必要がある

ポインタを使うことの代表的な利点

- 関数から戻値を(実質)複数受け取れる
- 関数に複数のデータを(=配列やファイルを)渡せる
- 動的メモリ割当てが可能になる

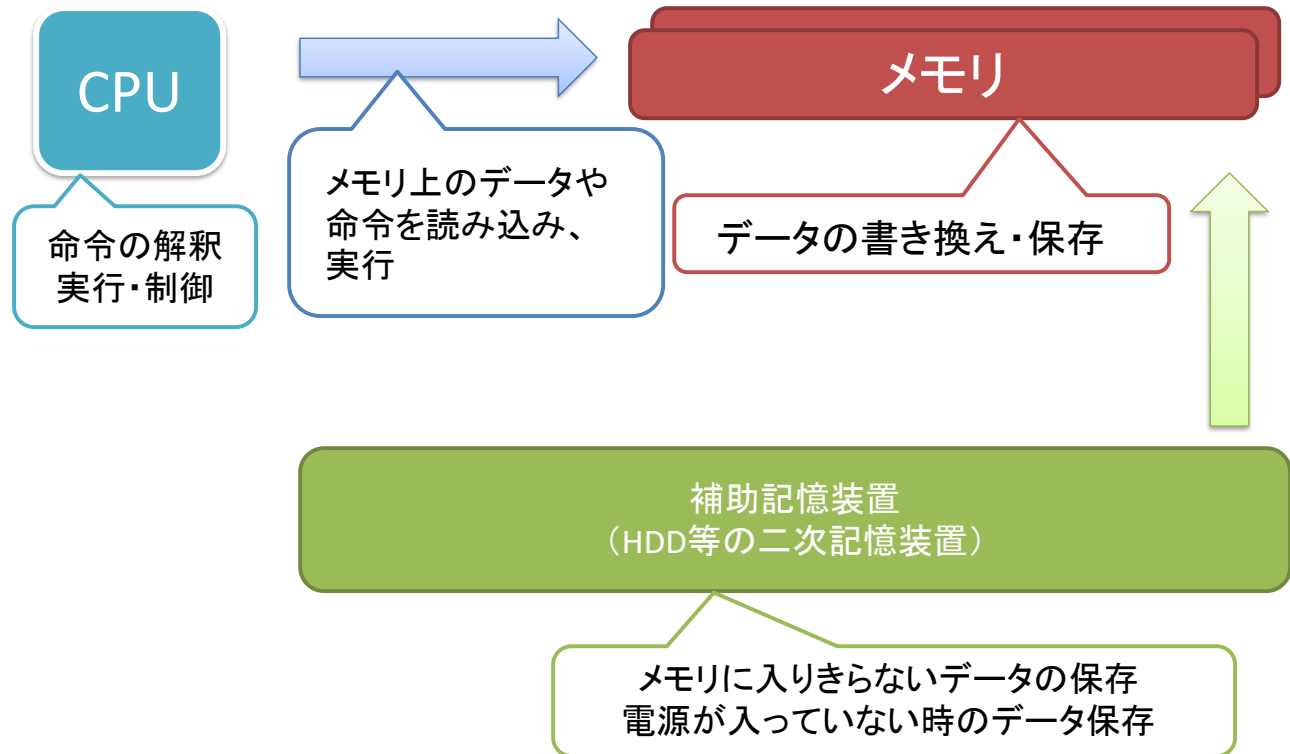
本演習で学ぶこと

- ポインタ変数の宣言・読み書きのための文法
- ポインタ渡しの文法
- 「値渡し」と「ポインタ渡し」の違い

14

2. ポインタ

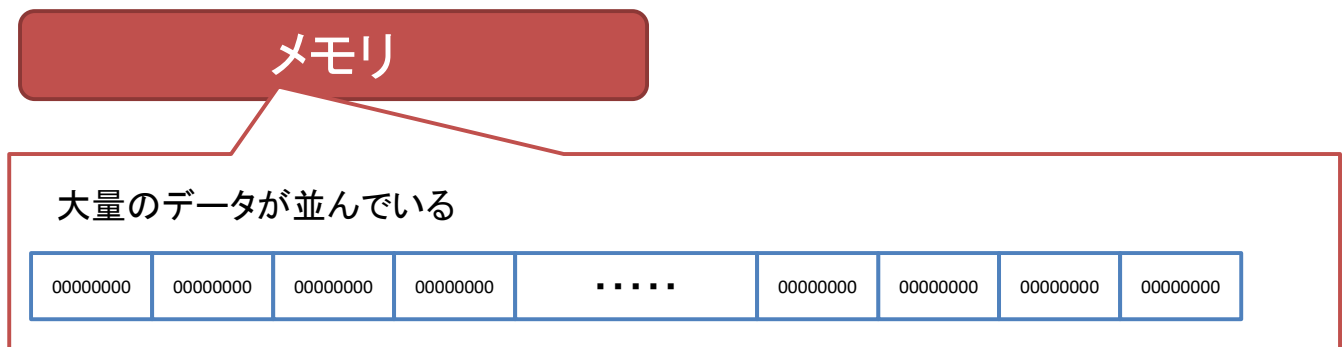
プログラムが動く仕組み



15

2. ポインタ

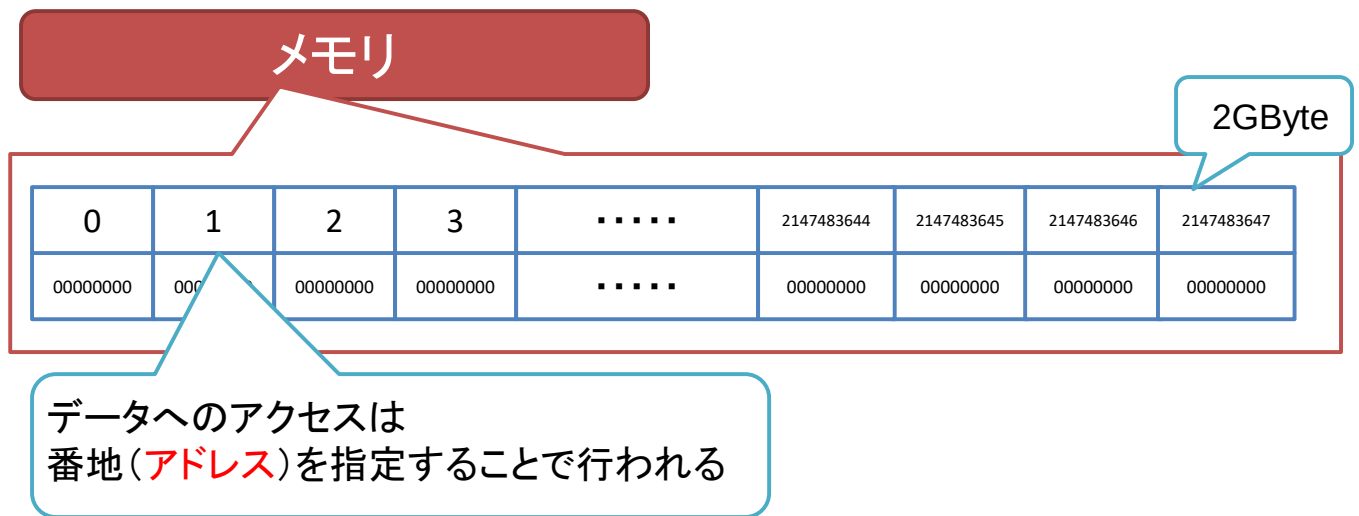
メモリの構造



16

2. ポインタ

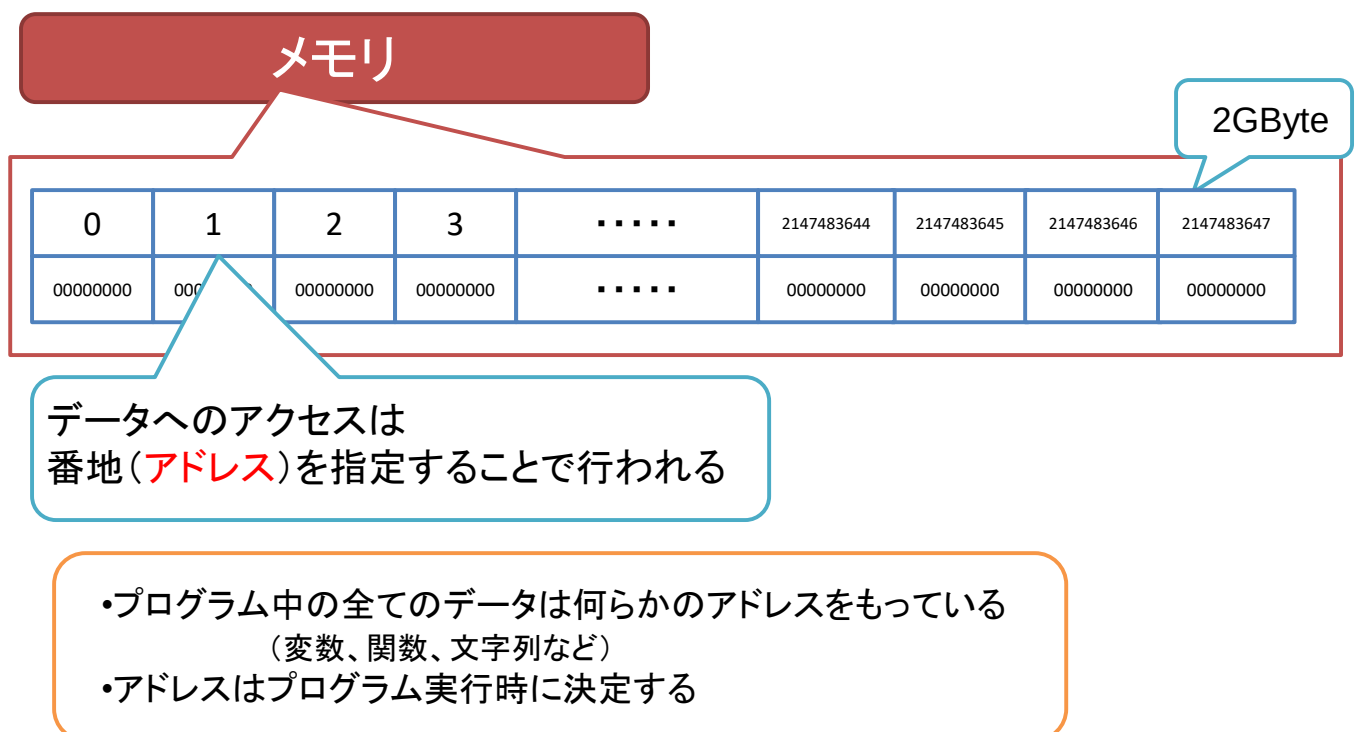
メモリの構造



17

2. ポインタ

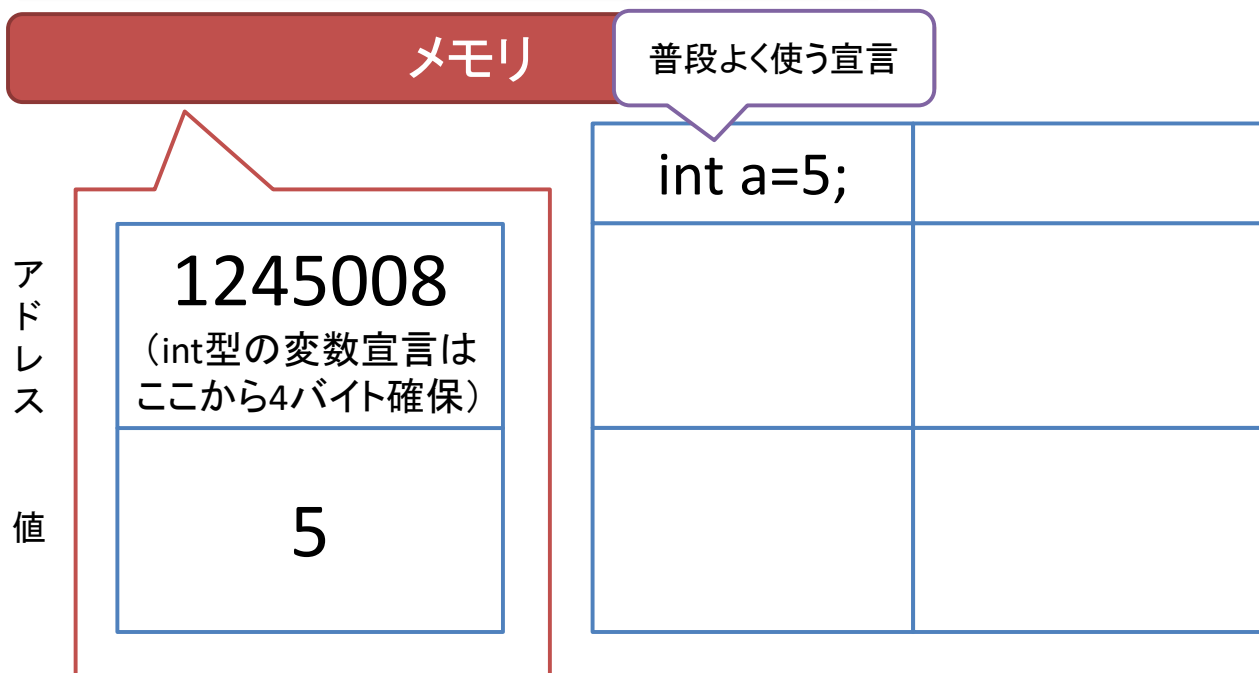
メモリの構造



18

2. ポインタ

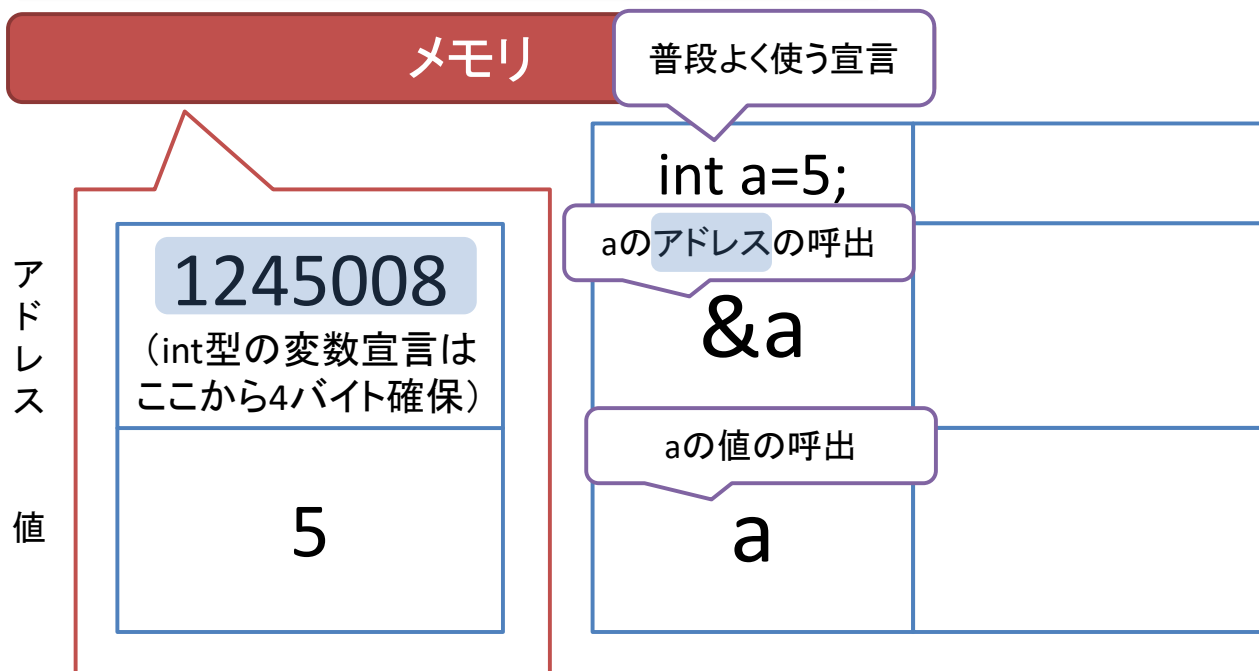
■ポインタ型(プログラム内での表現)



19

2. ポインタ

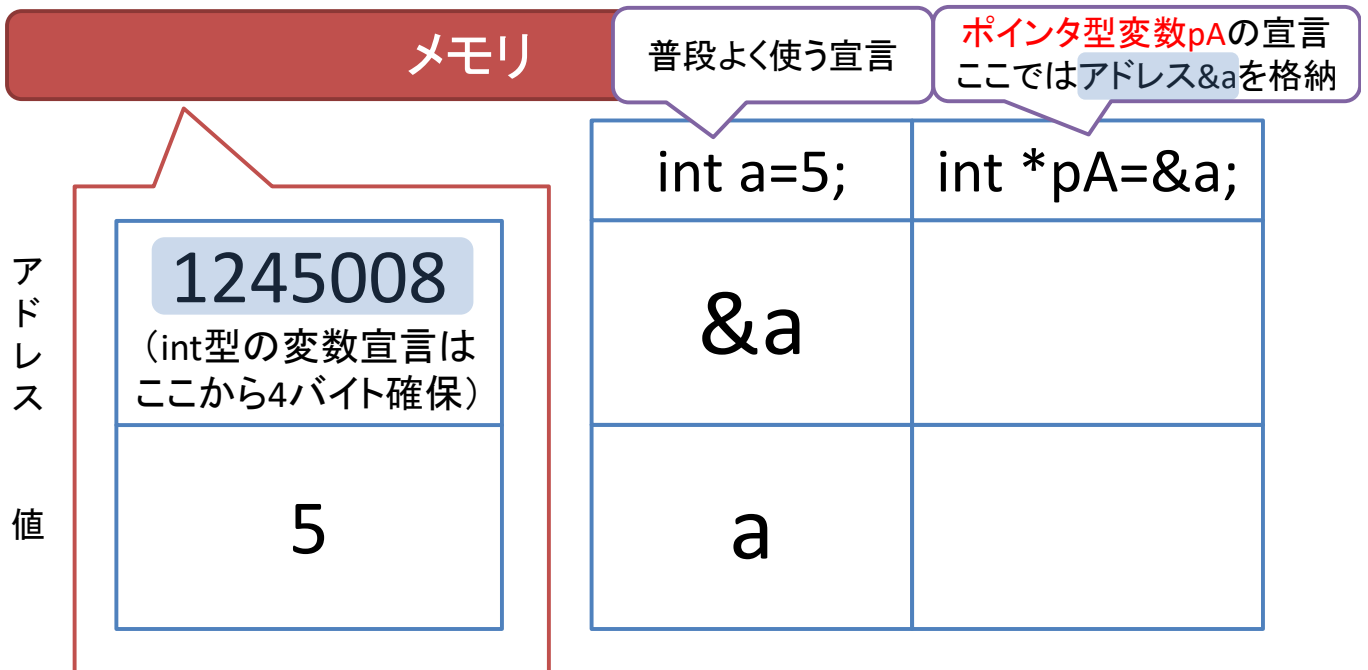
■ポインタ型(プログラム内での表現)



20

2. ポインタ

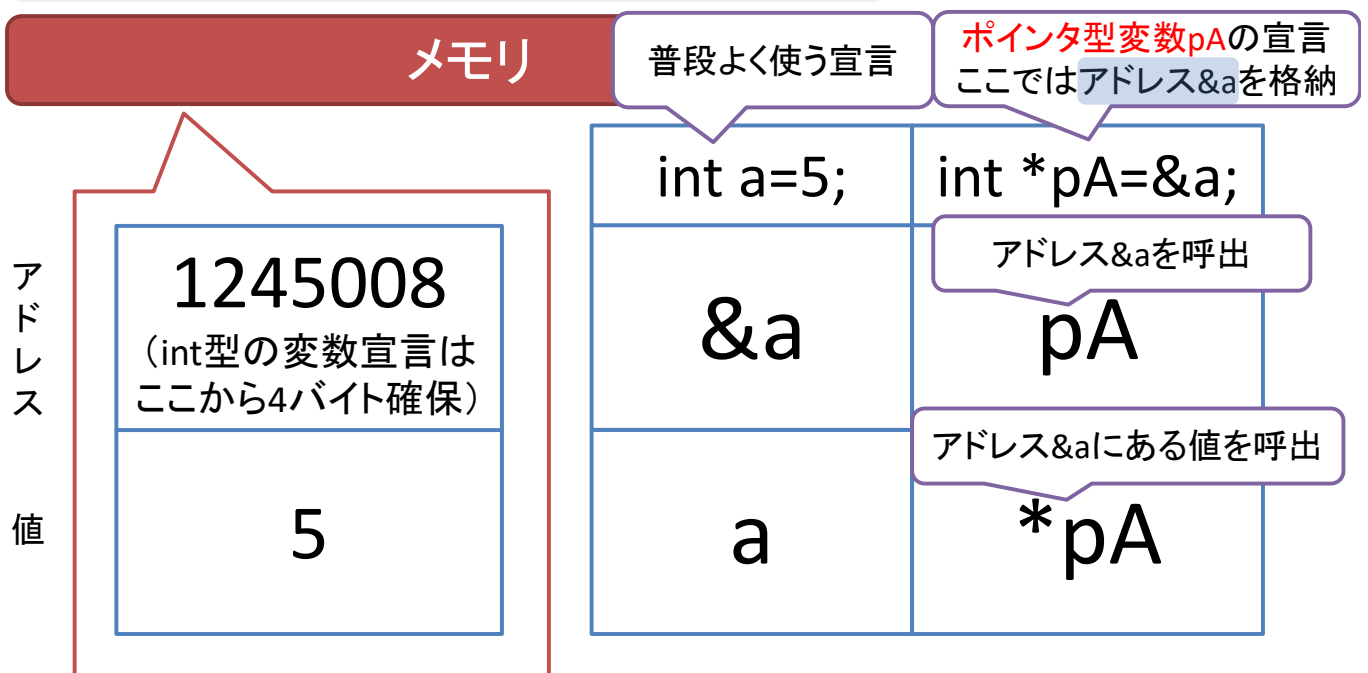
■ ポインタ型 (プログラム内での表現)



21

2. ポインタ

■ ポインタ型 (プログラム内での表現)



22

2. ポインタ

アドレス	&a	=	pA
値	a		*pA

■ ポインタ型変数

- アドレスを保存するための変数
- メモリ上のアドレスを利用したデータのやりとりができる

ポインタを使ったプログラムの例(※実際に動かすときには#include<stdio.h>を付けること)

```
int main(void){
    int a=5;
    int *pA;

    pA = &a;
```

実行結果(※aのアドレスやpAの値は環境によって異なる)
変数aの値は5です。
変数aのアドレスは0x7ffff1c2debc = 140737249468092です。
ポインタpAの値は0x7ffff1c2debc = 140737249468092です。
ポインタpAが示すアドレスにある値は5です。

```
    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, &a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);
    return 0;
```

```
}
```

23

2. ポインタ

アドレス	&a	=	pA
値	a		*pA

■ ポインタ型

ポインタ型の変数「pA」の宣言

ポインタ型では、そのアドレスに「何型の変数」があるのかを識別するために

「 **型名** ***ポインタ変数名;**」

の形式で宣言します

```
int main(void){
    int a=5;
    int *pA;

    pA = &a;
```

int *pA = &a;
と同じ

実行結果(※aのアドレスやpAの値は環境によって異なる)
変数aの値は5です。
変数aのアドレスは0x7ffff1c2debc = 140737249468092です。
ポインタpAの値は0x7ffff1c2debc = 140737249468092です。
ポインタpAが示すアドレスにある値は5です。

```
    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, &a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);
    return 0;
```

```
}
```

24

2. ポインタ

アドレス	&a	= pA
値	a	*pA

■ ポインタ型

アドレスは整数値(%pはアドレスの16進数表示)

「*ポインタ変数名」は、そのポインタが指すアドレスに格納されているデータ(値)を表現する。

```
int main(void){
    int a=5;
    int *pA;

    pA = &a;
```

実行結果
変数aの値は5です。
変数aのアドレスは0x7ffff1c2debc = 140737249468092です。
ポインタpAの値は0x7ffff1c2debc = 140737249468092です。
ポインタpAが示すアドレスにある値は5です。

```
    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, &a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);
    return 0;
```

Tips:%luを使うと&aやpAとは型が異なるためにwarningが出る。

(unsigned long int)を用いてキャストすればwarningは出なくなる→演習7.1参照

25

2. ポインタ

ポインタの有効な場面

複数の結果を返す関数

```
void func(int *p1,int *p2)
{
    *p1=1;
    *p2=2;
}
int main(void){
    int a,b;
    func(&a,&b);
    printf("a=%d¥nb=%d¥n",a,b);
    return 0;
}
```

実行結果

a=1
b=2

2. ポインタ

ポインタの有効な場面

複数の結果を返す関数

実行結果

```
void func(int *p1,int *p2)
```

```
{
```

```
    *p1
```

```
    *p2
```

```
}
```

```
int main(void)
```

```
    int a,b;
```

```
    func(&a,&b);
```

```
    printf("a=%d¥nb=%d¥n",a,b);
```

```
    return 0;
```

```
}
```

変数「a」、「b」用のメモリ領域が
確保される

メモリ上の様子

アドレス	変数名	データ
100	a	????
101	b	????
102		
103		
104		
105		

27

2. ポインタ

ポインタの有効な場面

複数の結果を返す関数

実行結果

```
void func(int *p1,int *p2)
```

```
{
```

```
    *p1=1;
```

```
    *p2
```

```
}
```

```
int main()
```

```
    int a,b;
```

```
    func(&a,&b);
```

```
    printf("a=%d¥nb=%d¥n",a,b);
```

```
    return 0;
```

```
}
```

変数「a」、「b」のアドレスが
関数「func」に渡される

メモリ上の様子

アドレス	変数名	データ
100	a	????
101	b	????
102		
103		
104		
105		

28

2. ポインタ

ポインタの有効な場面

複数の結果を返す関数

```
void func(int *p1,int *p2)
```

```
{
```

```
    *p1=1;
```

```
    *p2=2;
```

```
}
```

```
int main()
```

ポインタ「p1」、「p2」がメモリ上に確保され、変数「a」、「b」のアドレスがそれぞれ渡される

```
    func(&a,&b);
```

```
    printf("a=%d¥nb=%d¥n",a,b);
```

```
    return 0;
```

```
}
```

実行結果

メモリ上の様子

アドレス	変数名	データ
100	a	????
101	b	????
102	p1	100
103	p2	101
104		
105		

29

2. ポインタ

ポインタの有効な場面

複数の結果を返す関数

```
void func(int *p1,int *p2)
```

```
{
```

```
    *p1=1;
```

```
    *p2=2;
```

```
}
```

```
int main(void){
```

ポインタ「p1」、「p2」に格納されているアドレスに数値を代入

```
    func(&a,&b);
```

```
}
```

実行結果

メモリ上の様子

アドレス	変数名	データ
100	a	1
101	b	2
102	p1	100
103	p2	101
104		
105		

30

2. ポインタ

ポインタの有効な場面

複数の結果を返す関数

```
void func(int *p1,int *p2)
{
    *p1=1;
    *p2=2;
}
int main(void){
    int a,b;
    func(&a,&b);
    printf("a=%d\nb=%d\n",a,b);
    return 0;
}
```

アドレスを共有している変数「a」、「b」には、関数により書き換えられた結果が保存されており、それらの値を出力

実行結果

a=1
b=2

メモリ上の様子

アドレス	変数名	データ
100	a	1
101	b	2
102	p1	100
103	p2	101
104		
105		

31

2. ポインタ

関数「scanf」とアドレス

- 複数の数値を読み込み
- 変数に保存する関数

```
int main(void){
    int a,b;
    scanf("%d %d", &a,&b);
    printf("a=%d\n b=%d\n",a,b);
    return 0;
}
```

「scanf」の場合だけ「&」が必要な理由がわかっただろうか？

(戻り値無しで)変数の値を更新するために、アドレスを渡している

32

ここから演習課題です

33

3. 演習課題

演習7-1 ポインタ変数の利用

このプログラムをそのまま
実行してみよう

```
#include<stdio.h>
int main(void){
    int a=5;
    int *pA;

    pA = &a;

    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, (unsigned
        long int)&a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, (unsigned
        long int)pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);

    return 0;
}
```

34

3. 演習課題

演習7-2 値渡しとポインタ渡し確認＋ポインタ変数の表現

```
#include<stdio.h>
void swap0(int a, int b){
    int tmp;
    tmp = a; a = b; b = tmp;
}
void swap1(int *a, int *b){
    int tmp;
    tmp = *a; *a = *b; *b = tmp;
}
int main(void){
    int a, b;

    a = 1; b = 2;
    swap0(a, b);
    printf("aの値は%d, bの値は%d\n", a, b);
    a = 1; b = 2;
    swap1(&a, &b);
    printf("aの値は%d, bの値は%d\n", a, b);

    return 0;
}
```

このプログラムをそのまま
実行してみよう

35

3. 演習課題

演習7-3 ポインタ変数を用いたプログラムの作成

main関数内に変数aとbを宣言し、aとbの値をそれぞれa/bの商と余りに変更する、void型の関数divAndRemainderを実装せよ。ただし、aとbの値は整数である。

【禁止事項】

- ✕ main関数内での商と余りの計算
 - ✕ divAndRemainder内での式による出力
- 例) printf("%d %d\n", a/b, a%b); ← これはダメ

演習7-2ではaとbの値を交換する関数swap1を使いました。
それを上記を満たすように、うまく書き換えてみよう

36

発展7-1 入力するデータ数を動的に変更できるプログラム

要求される機能

1. データの件数を実行時に入力できる
2. 入力に応じてデータを受け取る
3. 好きな番号のデータを表示できる

※完全に理解するためには、ポインタ変数と配列の関係(次回)を学ぶ必要があります。

プログラムの例 `#include <stdlib.h>` が必要

```
int main(void){
    int DataSize;
    int *Data;
    printf("データの件数は?:");
    scanf("%d",&DataSize);
    Data = (int*)malloc(DataSize*sizeof(int));
    /*データの入力*/
    /*データの表示*/
    free(Data);
    return 0;
}
```

int型の値を
DataSize分格納
できるようにする

動的メモリ確保: malloc関数

関数「malloc」は引数にとるサイズ分のメモリをプログラムの実行中に確保する

戻り値は「先頭のアドレス」なので、この場合、「Data」番地から「n」個分の「int型」の大きさのメモリを自由に使える

※件数を10にした場合「int Data[10];」と宣言したことと同じになります。

不要になったら関数「free」で破棄する

37

本日の説明内容はおしまいです。
課題に取り組んでください。